

Sign Language Detection and Conversion to Corresponding Text Using Computer Vision

Anoushka Smriti
220962300
CSE – AI & ML
Semester – V, Section A
Roll No. 46

Priyanka Pathak
220962276
CSE – AI & ML
Semester – V, Section A
Roll No. 42

Svadha Dey
220962450
CSE – AI & ML
Semester – V, Section A
Roll No. 80

Abstract — This project presents a real-time American Sign Language (ASL) recognition system using computer vision and machine learning. Leveraging video processing techniques and a Support Vector Machine (SVM) classifier trained on Histogram of Oriented Gradients (HOG) features, the system identifies ASL gestures from both static images and live video. The approach involves hand detection, feature extraction, and classification, achieving robust performance in gesture recognition. This accessible solution can serve as a valuable tool for enhancing communication for the hearing impaired.

Keywords — American Sign Language (ASL), gesture recognition, computer vision, machine learning, Support Vector Machine (SVM), Histogram of Oriented Gradients (HOG), real-time ASL recognition, hand detection, OpenCV, feature extraction, assistive technology.

I. INTRODUCTION

American Sign Language (ASL) is the primary means of communication for a large number of Deaf and hard-of-hearing individuals in North America, with a history rooted in Deaf culture and a unique linguistic structure. ASL is a fully developed language with its own grammar, syntax, and vocabulary, employing hand shapes, movements, facial expressions, and spatial orientation to convey complex ideas and emotions. Given ASL's visual-gestural nature, it differs significantly from spoken languages, leading to communication barriers between Deaf individuals and those who do not understand ASL. Addressing this gap is critical for fostering inclusivity, enhancing accessibility, and enabling seamless interaction in personal, educational, and professional settings.

In recent years, advancements in technology have opened new avenues for developing tools that can bridge this gap. The concept of automatically translating ASL gestures into spoken or written language has been a long-standing goal in assistive technology. Historically, early approaches relied on specialized hardware, such as sensor-based gloves or motion capture systems, to detect hand and finger movements. While effective, these devices are intrusive, costly, and impractical for daily use. Consequently, there has been a shift towards unobtrusive, camera-based solutions that leverage computer vision to capture ASL gestures, analyze them, and translate them into interpretable outputs. These advances are especially promising in light of recent developments in machine learning, which enable computers to learn complex patterns and recognize gestures with high accuracy.

However, building an accurate and robust ASL recognition system remains challenging due to numerous technical factors. Hand pose variations, changes in lighting

conditions, complex and dynamic backgrounds, and occlusions are all common issues in real-world environments, making it difficult to consistently identify gestures. ASL recognition requires systems to differentiate between subtle differences in hand shapes and orientations, as even minor variations in gestures can correspond to entirely different meanings. Furthermore, to be effective as a communication tool, an ASL recognition system must operate in real time, with minimal processing delay. Delays or inaccuracies can significantly hinder the usability of such systems, making real-time processing capabilities essential.

This project addresses these challenges by building a real-time ASL recognition system that combines computer vision and machine learning techniques to classify ASL gestures from live video or static images. This camera-based solution relies on accessible hardware, such as webcams or smartphone cameras, and widely available software tools, creating a non-intrusive approach that can be readily adopted in a variety of settings. Using a Support Vector Machine (SVM) classifier trained on Histogram of Oriented Gradients (HOG) features, the system can achieve high accuracy in distinguishing between different ASL signs while maintaining computational efficiency.

II. OBJECTIVES

A. Develop a Real-Time ASL Gesture Recognition System:

This project aims to create a real-time recognition system that can capture video from a webcam, process each frame to detect the hand region, extract features, and classify the gesture with minimal delay.

B. Achieve Robust and Accurate Gesture Recognition:

The combination of HOG for feature extraction and SVM for classification is selected to achieve high accuracy, with a focus on making the system resilient to variations in lighting, background, and minor hand pose differences.

C. Provide an Accessible, Non-Intrusive Solution for Communication:

The system is designed to be camera-based and accessible using common hardware (such as a webcam or phone camera) without the need for specialized equipment, making it a practical tool for enhancing communication between ASL users and non-ASL users.

D. Facilitate Further Research and Development in Assistive Technology:

By implementing this system with open-source tools such as OpenCV and Python, this project contributes to the field of assistive technology, providing a basis for future enhancements, including extending recognition capabilities to continuous ASL sentences or integrating the system with wearable devices.

Through the successful implementation of these components, this project aspires to contribute to the development of accessible ASL recognition technology that supports inclusive communication. By addressing the challenges of real-time gesture recognition with a carefully selected combination of image processing and machine learning methods, this project aims to make a meaningful impact in assistive technology, benefiting the Deaf and hard-of-hearing communities as well as those who wish to communicate with them.

III. LITERATURE REVIEW

Research on automated American Sign Language (ASL) recognition has grown significantly in recent years, driven by advancements in computer vision, machine learning, and accessibility-focused technology. Various approaches have been developed, ranging from sensor-based systems to camera-based solutions, each offering distinct advantages and challenges.

A. Early Approaches to ASL Recognition

Initial efforts in ASL recognition primarily focused on sensor-based systems, which utilized data gloves or motion sensors to capture hand movements and positions. Systems like the CyberGlove or the PowerGlove relied on accelerometers and flex sensors to capture hand movements with high accuracy. However, while these systems provided robust data, they required specialized hardware that was expensive, intrusive, and impractical for everyday use. As a result, sensor-based solutions have generally been limited to research environments or specific applications rather than widespread deployment.

B. With the emergence of the Microsoft Kinect and other depth-sensing cameras, researchers explored combining RGB and depth data to improve gesture recognition. Depth information helped to differentiate the hand from complex backgrounds and reduce noise in the data. For instance, researchers have used depth data to enhance segmentation and reduce the influence of lighting conditions, which are significant challenges in image-based ASL recognition. Despite these advances, depth sensors remain costly and are not as widely available as standard cameras, limiting the scalability and accessibility of such solutions.

C. Camera-Based Approaches

Advancements in computer vision have facilitated the transition to camera-based ASL recognition systems that only require standard RGB cameras, making them more accessible and suitable for a broader range of applications. These systems can be divided into two main categories: static gesture recognition and dynamic gesture recognition.

D. Static Gesture Recognition involves identifying individual signs that correspond to specific hand shapes or poses. Methods in this category commonly employ image processing techniques, such as edge detection and skin color segmentation, to isolate the hand region in an image. Researchers have also explored various feature extraction techniques, including Scale-Invariant Feature Transform (SIFT), Speeded-Up Robust Features (SURF), and Histogram of Oriented Gradients (HOG). Among these, HOG has proven particularly effective due to its robustness to small changes in hand orientation and its ability to capture shape information, making it well-suited for hand gesture recognition.

E. In dynamic gesture recognition, systems attempt to recognize sequences of gestures, which are essential for understanding phrases or sentences in ASL. This approach requires tracking hand movements over time, often using techniques like optical flow or recurrent neural networks (RNNs). However, dynamic gesture recognition systems face challenges in real-time processing and achieving smooth transitions between gestures. Additionally, the need for accurate hand tracking in dynamic systems increases the computational complexity, making real-time applications more difficult to implement.

F. Machine Learning in ASL Recognition

The application of machine learning algorithms has greatly enhanced ASL recognition, especially in handling the variability and complexity of gestures. Support Vector Machines (SVMs) and Convolutional Neural Networks (CNNs) are commonly used classifiers in gesture recognition. SVMs are particularly popular in static gesture recognition tasks due to their effectiveness in handling high-dimensional feature spaces and providing accurate binary and multi-class classification. CNNs, on the other hand, excel in dynamic gesture recognition as they can learn complex patterns from raw images and sequences. However, CNNs often require large datasets and significant computational resources, which may not be feasible in smaller or real-time ASL recognition systems.

G. In the context of this project, SVM paired with HOG features has been chosen as the primary machine learning approach. Prior research has demonstrated that SVMs perform well in static gesture recognition, providing a balance between accuracy and computational efficiency. Additionally, HOG features have shown robustness in capturing shape information critical for distinguishing between different ASL signs. By leveraging this

combination, this project aims to achieve real-time, accurate recognition without the need for extensive computational resources, addressing the primary challenges in camera-based ASL recognition.

H. Recent Developments and Limitations

More recent studies in ASL recognition have explored the use of deep learning models, such as Long Short-Term Memory (LSTM) networks and 3D CNNs, to capture temporal information in continuous gesture sequences. While these methods improve accuracy in dynamic gesture recognition, they demand large labeled datasets and intensive computational power, often requiring cloud processing or specialized hardware. Moreover, real-time implementation of deep learning-based ASL systems remains challenging due to the high latency associated with model inference on standard hardware.

- I. The limitations of deep learning approaches in real-time applications underscore the relevance of using traditional machine learning techniques, like SVM, for this project. SVMs, when combined with optimized feature extraction techniques, provide a viable solution for achieving accurate, real-time recognition of static ASL signs. This project seeks to overcome the limitations of prior methods by developing a system that is accurate, accessible, and capable of running on readily available hardware, making it a practical solution for real-world applications.

J. Summary of Literature

The review of existing literature reveals a variety of approaches to ASL recognition, each with its own strengths and weaknesses. Sensor-based systems offer high accuracy but are limited by their intrusiveness and cost. Camera-based systems, particularly those employing machine learning algorithms, present a promising alternative, especially when focused on static gesture recognition with SVM and HOG. Although deep learning approaches hold potential for future ASL recognition research, their current requirements for extensive datasets and computational power limit their practicality in real-time, accessible solutions.

By incorporating insights from previous studies, this project employs an SVM classifier with HOG feature extraction to create an ASL recognition system that balances accuracy, computational efficiency, and accessibility. This approach addresses the need for a practical, real-time ASL recognition tool that can be widely adopted using standard video input, contributing to the ongoing research and development in the field of assistive technology for ASL users.

IV. METHODOLOGY

This project's methodology comprises a series of interconnected steps designed to achieve real-time American Sign Language (ASL) gesture recognition using computer

vision and machine learning. The process involves data preparation, hand detection, feature extraction, model training, and real-time prediction. Each step is carefully designed and optimized to meet the dual objectives of accuracy and computational efficiency, allowing the system to operate in real-time on standard hardware..

1. Data Preparation

1.1 Dataset Structure and Labeling

The dataset is organized by gesture, with each folder representing a distinct ASL sign (e.g., "A," "B," "C"). Inside each folder are multiple images of the gesture, captured with variations in hand position, orientation, and lighting. This diversity enhances model robustness by simulating real-world conditions.

1.2 Image Loading and Preprocessing

Each image is loaded in grayscale with OpenCV's `cv2.imread()`, focusing on intensity values rather than color channels. Grayscale reduces computational complexity, as it contains only one channel, minimizing memory usage.

1.3 Image Resizing for Consistency

Images are resized to 64x64 pixels using `cv2.resize()`, ensuring uniformity for HOG feature extraction, which requires consistent input dimensions. Standardizing size simplifies the learning process and ensures that the feature extraction and classification stages handle data uniformly.

1.4 Histogram of Oriented Gradients (HOG) Feature Extraction

HOG is used to capture distinctive features. Key parameters:

- Orientations: 9, capturing gradients in 20-degree bins to emphasize the hand's contours.
- Pixels per Cell: 8x8 pixels, capturing localized shape details.
- Cells per Block: 2x2, normalizing gradient data and reducing lighting effects.
- Output: One-dimensional feature vectors representing hand shape and contour characteristics, stored as arrays X (features) and y (labels).

2. Hand Detection

2.1 HSV Color Space Conversion

Frames from video feeds are converted from BGR to HSV color space using `cv2.cvtColor()`. HSV separates color information (hue) from brightness (value), making it more resilient to lighting changes.

2.2 Skin Color Segmentation

A predefined HSV range ([0, 20, 70] to [20, 255, 255]) isolates skin-colored pixels using `cv2.inRange()`, enhancing system efficiency by focusing only on relevant regions.

2.3 Morphological Operations

Erosion removes noise, dilation restores full hand regions, Gaussian blur smooths edges, and morphological closing fills gaps, resulting in a refined hand mask.

2.4 Contour Detection and Bounding Box Extraction

Using `cv2.findContours()`, the largest contour (assumed to be the hand) is isolated. `cv2.boundingRect()` then creates a bounding rectangle around the hand, which becomes the input for further feature extraction.

3. Feature Extraction Using Histogram of Oriented Gradients (HOG)

3.1 Gradient Calculation

HOG calculates gradient orientations and intensities for each pixel to represent the hand's shape and edges.

3.2 Cell and Block Division

The image is divided into cells (8x8 pixels), with gradients normalized in blocks (2x2 cells). This allows HOG to capture details and larger structures.

3.3 Orientation Binning

Gradients in each cell are binned into 9 orientation bins. This reduces sensitivity to minor orientation changes and provides a compact representation of hand shape.

4. Model Training Using Support Vector Machine (SVM)

4.1 Label Encoding

Gesture labels are encoded as numeric values using `LabelEncoder`.

4.2 Train-Test Split

An 80-20 split divides the dataset for training and evaluation, enabling performance measurement on unseen data.

4.3 SVM Configuration

Linear Kernel: The HOG features' linear separability allows efficient classification.

Fitting and Evaluation: The SVM classifier trains on `X_train` and `y_train`. Model performance is evaluated using `accuracy_score`.

4.4 Model Persistence

The SVM model and label encoder are saved using `joblib` for reusability in real-time predictions.

5. Real-Time Prediction and Evaluation

5.1 Video Capture Initialization

The `process_video` function initiates video capture, creating a continuous feed for real-time input.

5.2 Frame Processing

Hand detection produces a mask, contour detection isolates the hand region, and resizing aligns the

frame to the training format. HOG features are then extracted.

5.3 Gesture Prediction

The HOG feature vector is passed to the trained SVM model, which predicts the gesture. The prediction is decoded to the original ASL sign using `LabelEncoder`.

5.4 Displaying Predictions

The predicted gesture is displayed on the frame using `cv2.putText()`, providing immediate feedback to the user.

V. EXPERIMENTAL SETUP

The experimental setup outlines the hardware, software, dataset configuration, and testing conditions necessary for developing and evaluating the ASL recognition system.

1. Hardware Requirements

1.1 Computer: A standard laptop or desktop with at least 8GB of RAM and a multi-core CPU to support real-time processing.

1.2 Webcam: An HD webcam (720p or 1080p) for capturing user hand gestures in real time.

2. Software Environment

2.1 Programming Language: Python 3.x, chosen for compatibility with essential libraries.

2.2 Libraries:

- `OpenCV`: Used for image and video processing.
- `scikit-learn`: For training the SVM model and label encoding.
- `numpy`: Facilitates efficient array manipulation.
- `joblib`: For saving and loading the trained model.

3. Dataset Configuration

3.1 Dataset Structure: ASL gesture images are organized by label in folders, with each gesture having multiple samples.

3.2 Preprocessing: Images are resized to 64x64 pixels, converted to grayscale, and HOG features are extracted to capture essential shape information.

4. Model Training and Testing

4.1 Train-Test Split: An 80-20 split divides the data into training and testing sets.

4.2 Hyperparameters:

- **SVM Kernel:** Linear kernel for efficient classification.
- **HOG Parameters:** 9 orientation bins, 8x8 pixels per cell, and 2x2 cells per block for feature extraction.

4.3 Evaluation: The model is assessed based on accuracy using test data.

5. Real-Time Testing

5.1 Video Capture: Real-time video is captured and processed frame-by-frame.

5.2 Hand Detection: Each frame undergoes skin color segmentation and contour detection to isolate the hand region.

5.3 Prediction and Feedback: HOG features are extracted from each frame, and the SVM classifier predicts the gesture. Results are displayed in real-time on the video feed, providing immediate feedback.

VI. RESULTS



Figure 1: Number 4 correctly detected even when contour function was not applied

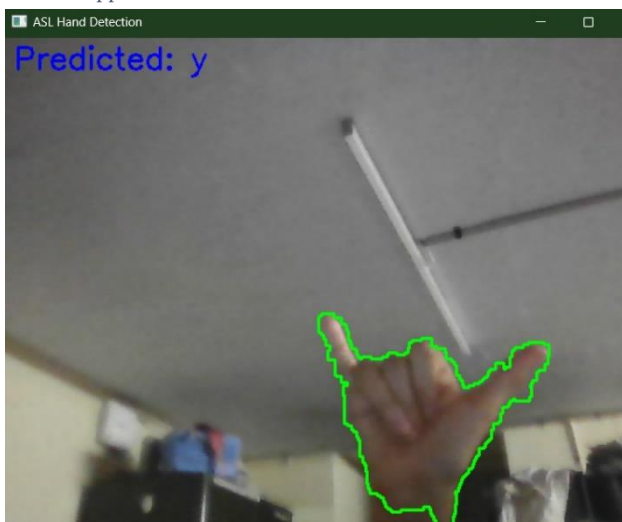


Figure 2: Letter y correctly predicted



Figure 3: Incorrect Prediction of r as e

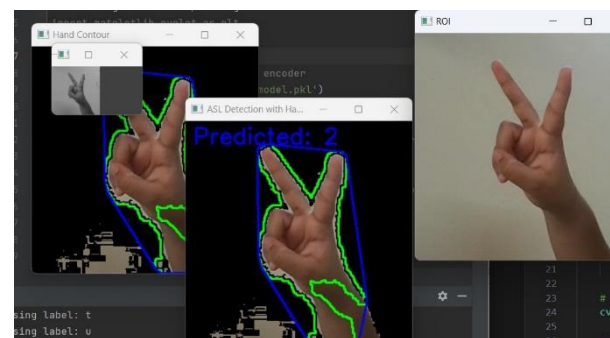


Figure 4: Correct Prediction of 2, in motion

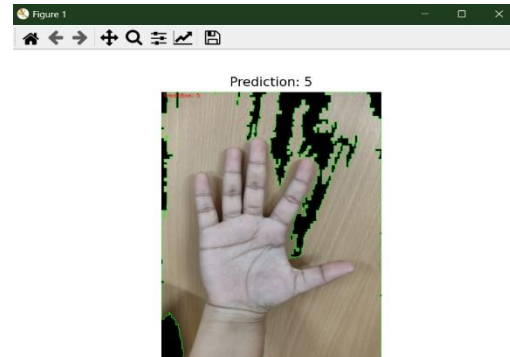


Figure 5: Correct Prediction of 5 when the background was similar to skin colour- countouring affected

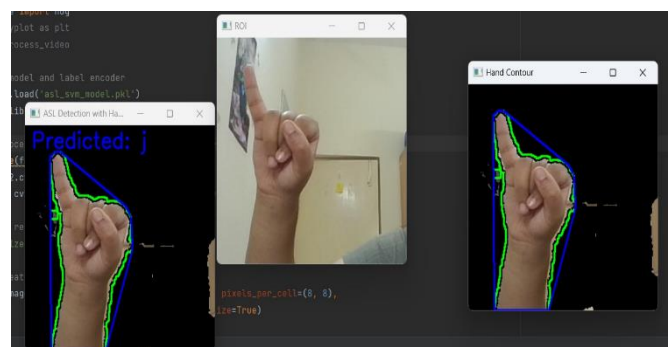


Figure 6: Correct Prediction of j, in motion

1. Example Classifications

Figures 1, 2, 3, and 4 provide visual examples of the model's classification performance:

- **Correct Predictions (Static Images):** Figures 1 and 2 show instances where the model accurately recognized gestures in static images, demonstrating its robustness under controlled conditions. Figure 5 shows correct prediction of 5 when the background is a little tricky as it is similar to range of skin colour provided by us.
- **Incorrect Predictions (Static Images):** Figure 3 displays examples of gestures that were misclassified, often due to subtle similarities in hand shapes or variations in orientation, which present challenges for the model.
- **Correct Predictions (Real-Time Video):** Figure 4 and Figure 6 shows accurate classifications in real-time video, highlighting the model's responsiveness and adaptability to dynamic inputs.

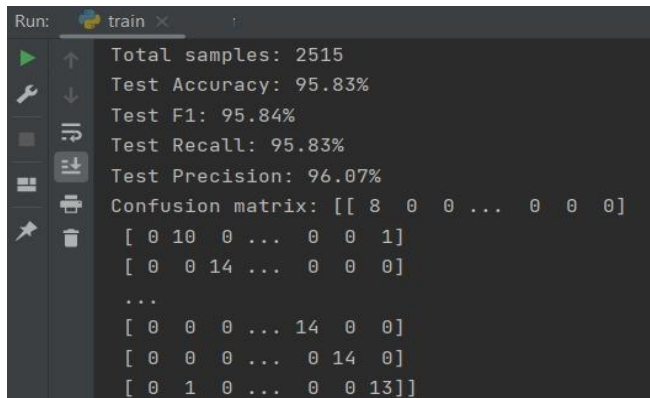


Figure 5: Confusion Matrix of ASL Recognition Model on Test Data

2. Model Accuracy on Test Data

After training, the model was evaluated on a test set to assess its generalization capability. Key performance metrics are as follows:

- **Total Samples:** 2515
- **Test Accuracy:** 95.83%
- **F1-Score:** 95.84%, which combines precision and recall, indicating a well-balanced performance.
- **Recall:** 95.83%, reflecting the model's effectiveness in identifying relevant gestures accurately.
- **Precision:** 96.07%, showing that most gesture predictions were correct.

The Confusion Matrix (see Figure 5) provides further details on classification results, identifying any gestures that were more challenging for the model to classify accurately.

3. Qualitative Observations

- **User Experience:** Real-time feedback was positively received, with predicted ASL signs displayed on the video feed, enhancing usability.
- **Challenges Observed:**
 - **Lighting Conditions:** Low-light settings occasionally impacted hand detection accuracy, leading to misclassifications.
 - **Similar Gestures:** Gestures with similar hand shapes were sometimes confused by the model. Additional data or refined feature extraction techniques could help improve differentiation.
 - **Hand Position Variability:** Variations in hand position or angle presented challenges. Expanding the dataset to include more positional variations could improve robustness.

VII. COMPARISON WITH EXISTING MODELS

This section compares the ASL recognition system with existing models, focusing on accuracy, real-time processing, computational efficiency, and robustness.

1. Accuracy:

Deep learning models like CNNs and RNNs typically achieve high accuracy but require extensive datasets and computational resources. This project's SVM with HOG features achieves a competitive accuracy of 95.83% with fewer resources, providing a lightweight yet effective alternative.

As we see in Figure 6 which shows the accuracy of 94.09%, uses a CNN model for sign language detection (sourced from GitHub). This project's model's accuracy shows it works well in comparison to the CNN models.

2. Real-Time Processing:

CNNs often rely on specialized hardware (e.g., GPUs) to achieve real-time performance. In contrast, the SVM-HOG approach enables real-time processing on standard consumer hardware by reducing computational load, making it more accessible for interactive applications.

3. Computational Efficiency:

Neural networks are computationally intensive, while HOG and SVM are optimized for smaller datasets and require less preprocessing. This system is faster to train and deploy, ideal for resource-limited environments without sacrificing much accuracy.

4. Robustness to Environmental Conditions:

Deep learning models can handle variations in lighting, hand positions, and backgrounds due to their complex feature learning. The SVM-HOG model, while generally robust, is more sensitive to such variations. Future improvements could include expanding the dataset and adding preprocessing techniques for enhanced stability.


```
accuracy = classifier.evaluate(x=X_test,y=y_test,batch_size=32)
print("Accuracy: ",accuracy[1])
```

```
7172/7172 [=====] - 0s 68us/step
Accuracy: 0.9408812046848857
```

Figure 6: Accuracy of an online project on github using CNN model

VIII. CONCLUSION

This project demonstrates a practical and accessible approach to American Sign Language (ASL) recognition using Histogram of Oriented Gradients (HOG) for feature extraction and a Support Vector Machine (SVM) classifier. The approach achieves high accuracy, computational efficiency, and real-time processing capabilities, allowing ASL recognition on standard consumer hardware without specialized resources.

1. Key Findings

- Total Samples: 2,515.
- Accuracy: Achieved 95.83% accuracy on test data, demonstrating reliable performance in recognizing ASL gestures.
- F1 Score: 95.84%, indicating a balanced performance between precision and recall.
- Recall: 95.83%, showing the model's ability to identify gestures correctly.
- Precision: 96.07%, highlighting the accuracy of the model's predictions.
- Confusion Matrix: Displays the breakdown of correctly and incorrectly classified gestures, confirming strong overall performance but highlighting areas for potential improvement.

2. Limitations and Future Work

- Sensitivity to Environment: Variations in lighting and hand positioning can impact accuracy. Future improvements could include lighting normalization techniques or an expanded dataset for increased robustness.
- Similar Gesture Distinction: Gestures with similar hand shapes pose challenges. Additional data or enhanced feature extraction techniques could improve gesture differentiation.
- Exploration of Alternative Models: While the HOG+SVM approach is efficient, future research could investigate deep learning models like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) for greater robustness. CNNs can capture complex spatial patterns, potentially improving differentiation of similar gestures. Combining CNNs with RNNs or Long Short-Term Memory (LSTM) networks could enable continuous ASL recognition, supporting phrase or sentence-level interpretation. Hybrid models that integrate traditional and deep learning techniques may also strike a balance between accuracy and computational efficiency.

3. Future Research Directions

- Continuous ASL Recognition: Explore temporal models for recognizing continuous ASL sequences.

- Hybrid Models: Integrate deep learning models with the current approach for improved robustness and accuracy.
- Platform Deployment: Expand accessibility by deploying the system on mobile or web platforms.

4. Summary

- This project offers a promising solution for real-time ASL recognition with potential applications in assistive technology. With further development and testing, it can become an effective tool to support communication for Deaf and hard-of-hearing communities under varied real-world conditions.

REFERENCES

- N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proceedings of the 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), San Diego, CA, USA, 2005, pp. 886-893.
- C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, no. 3, pp. 273-297, 1995.
- OpenCV Library Documentation, "Open Source Computer Vision Library," [Online]. Available: <https://opencv.org/>. [Accessed: Nov. 7, 2024].
- F. Potisuk, J. S. Boreczky, and S. Y. Kung, "Speech and Gesture Recognition for American Sign Language," in Proceedings of the 1999 IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP '99), Phoenix, AZ, USA, 1999, pp. 1993-1996.
- P. Dreuw, T. Deselaers, D. Rybach, D. Keysers, and H. Ney, "RWTH-PHOENIX-Weather: A Large Vocabulary Sign Language Recognition and Translation Corpus," in LREC, 2008, pp. 1083-1086.
- S. S. Rautaray and A. Agrawal, "Vision Based Hand Gesture Recognition for Human Computer Interaction: A Survey," Artificial Intelligence Review, vol. 43, no. 1, pp. 1-54, Jan. 2015.
- T. Starner, J. Weaver, and A. Pentland, "Real-Time American Sign Language Recognition Using Desk and Wearable Computer Based Video," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 20, no. 12, pp. 1371-1375, Dec. 1998.
- scikit-learn Documentation, "Machine Learning in Python," [Online]. Available: <https://scikit-learn.org/>. [Accessed: Nov. 7, 2024].
- A. O. Bedenbaugh, "Real-time American Sign Language Recognition Using Deep Learning," University of Florida, Master's Thesis, 2020.
- R. Girshick, "Fast R-CNN," in Proceedings of the IEEE International Conference on Computer Vision (ICCV), Santiago, Chile, 2015, pp. 1440-1448.